PROJECT GUIDE · SORO NUA AGENCY, DOUALA

Soro Nua Campaign Predictor

The complete guide: what the project is, why it exists, how it was built step by step, and how everything works — explained for someone who has never seen it.

Written for: newcomers, supervisors, jury members and future maintainers
Project: internal advertising-performance prediction tool, built during an internship (22 June – 27 August 2026)
Intern: MEKA-MEVOUNG JEAN DAUNICK

Contents

1 The project in one page

2 The problem we are solving

3 What the tool does (a guided tour)

4 The key ideas, explained simply

5 The data

6 How the system is built (the big picture)

7 How every feature works, step by step

8 Security and privacy

9 Our approach: how the work was organised

10 The steps taken, phase by phase

11 Quality: how we know it works

12 Running and demonstrating the project

13 Honest limits and what comes next

14 Questions people ask

15 Glossary and map of the repository

How to read this guide

If you only have five minutes, read section 1. If you want to understand the idea, read sections 2 to 5. If you want to understand how it is built, continue with sections 6 to 8. Sections 9 to 11 tell the story of how the work was done, and section 12 explains how to run it yourself. Technical words are explained when they first appear and again in the glossary (section 15).

1 The project in one page

Soro Nua Campaign Predictor is a small web application that tells a marketing team **what an online advertising campaign is likely to achieve before any money is spent**.

Who it is for

The marketing team of **Soro Nua Agency**, an advertising agency in Douala, Cameroon. Two kinds of users: **marketers** (they describe campaigns) and **administrators** (they also manage users, data and the prediction model).

What you do with it

You describe a campaign (channel, type, budget, the kind of person you target). The tool answers with three predicted numbers, a **Low / Medium / High** rating, the reasons behind the numbers, and up to three changes that would improve the result.

3	8,000	≤ 3	100 %
numbers predicted: click rate, conversion rate, website visits	example campaigns used to teach the first model	suggested improvements per campaign	runs in Docker containers, one command to start

How it works in three sentences

1. A statistical model (a method called **XGBoost**) has **learned from thousands of past campaigns** how budget, channel, audience and so on relate to results.
2. When you describe a new campaign, the model **predicts** the results; a scoring rule turns them into a rating, and the system **tests alternatives** (another channel, another budget…) to propose improvements.
3. Everything sits behind a **secure sign-in**; each person only sees their own saved predictions, and administrators can add the agency's real data and train better versions of the model.

> **The most important honest fact**
>
> Because the agency does not yet have its own campaign history, the model learned from a **public, artificially generated dataset** in which results are almost unrelated to the inputs. The tool therefore predicts **little better than an average**, and says so openly on screen. The technical chain is validated; the predictions will only become meaningful once the agency imports its real campaign data. Section 5 and section 13 explain this in detail.

2 The problem we are solving

The situation today

Imagine the marketing team at Soro Nua preparing a new online campaign. They must decide, in advance: **how much to spend**, **which channel** to use (email, paid search, organic search, social media or referral), **what type of campaign** to run (awareness, consideration, conversion or retention) and **who to target**.

Today those choices rest on **experience and intuition**. That has three weaknesses:

No numbers

Intuition cannot say how many clicks, conversions or visits to expect, nor be checked afterwards.

Too late

The real results arrive only after the money is spent and the campaign has run.

No history

The agency has no structured data about its past campaigns to base decisions on.

What the agency asked for

The project brief (the first and highest-ranking document of the project) defines success with eight criteria. In plain language, a team member must be able to:

1. describe a campaign in an interactive English dashboard;
2. see three predicted numbers: click-through rate, conversion rate and website visits;
3. see an expected performance level: low, medium or high;
4. see which elements of the campaign weigh the most on those numbers;
5. get up to three suggested changes that would improve the result (or a clear statement that nothing helps);
6. change any value and see new predictions;
7. trust the numbers because they were **tested on data the model never saw**, with the reliability measured and displayed;
8. (administrator) import the agency's own data, retrain the model, and replace the old model only deliberately.

What the project deliberately does not do

Good scoping is as important as good building. The first version explicitly does not: serve other companies; predict anything other than the three numbers; advise on advertisement *content* or on a particular advertising platform (Facebook, Google…); pull data automatically from ad platforms; or offer any language other than English. It also does not use generative AI (such as Gemini) in this first version.

3 What the tool does (a guided tour)

The best way to understand the project is to follow a user through it. (The screenshots in this guide come from the interface prototype of the project; the numbers shown on them are illustrative.)

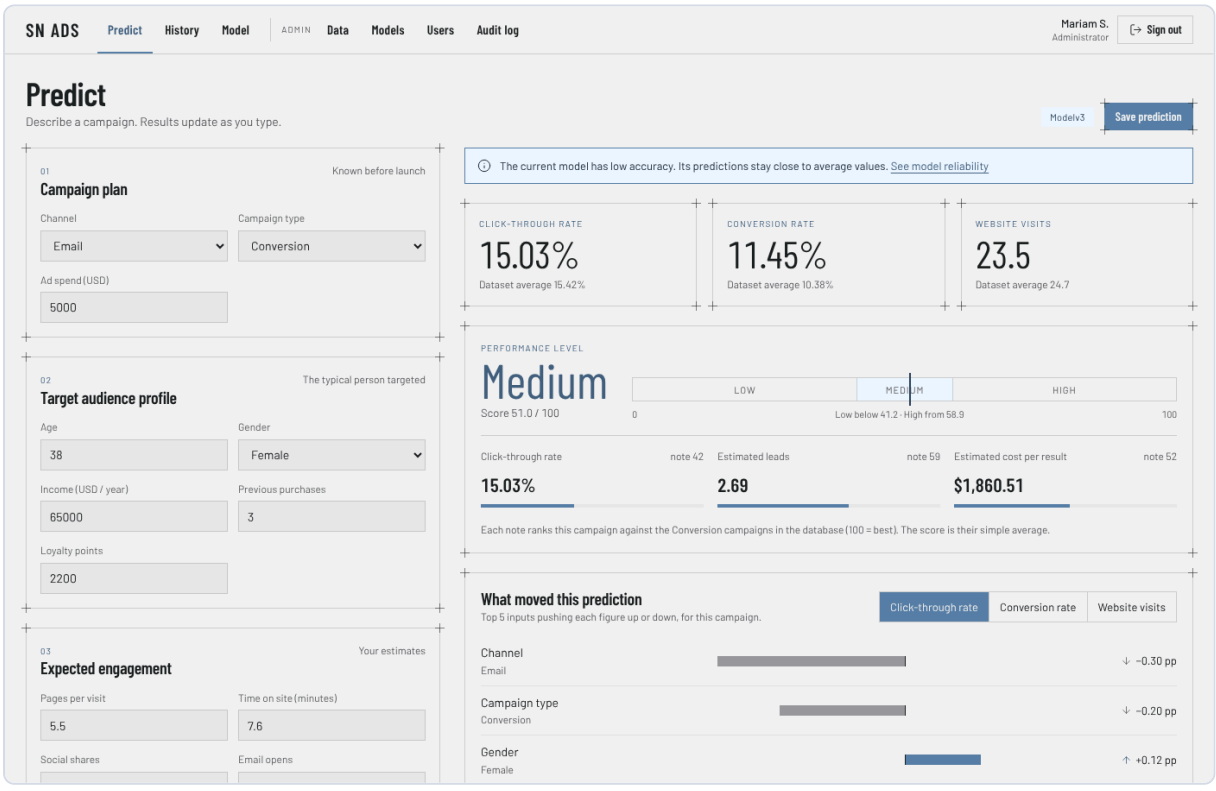
Step 1 — Sign in

There is no public sign-up. An administrator **invites** a colleague by e-mail; the colleague opens the link, chooses a password (at least 10 characters) and arrives on the main page. A wrong login always gives the same message, “Invalid email or password.”, so nobody can discover which e-mail addresses exist.

Step 2 — Describe a campaign (the “Predict” page)

The form has three blocks:

- **Campaign plan** — what the team decides before launch: channel, campaign type, budget (ad spend).
- **Target audience profile** — the “typical person” the campaign aims at: age, gender, income, number of past purchases, loyalty points.
- **Expected engagement** — what the team *estimates* will happen after launch: pages per visit, time on site, social shares, e-mail opens, e-mail clicks. These fields are pre-filled with the average observed for the same channel and campaign type, and can be edited.



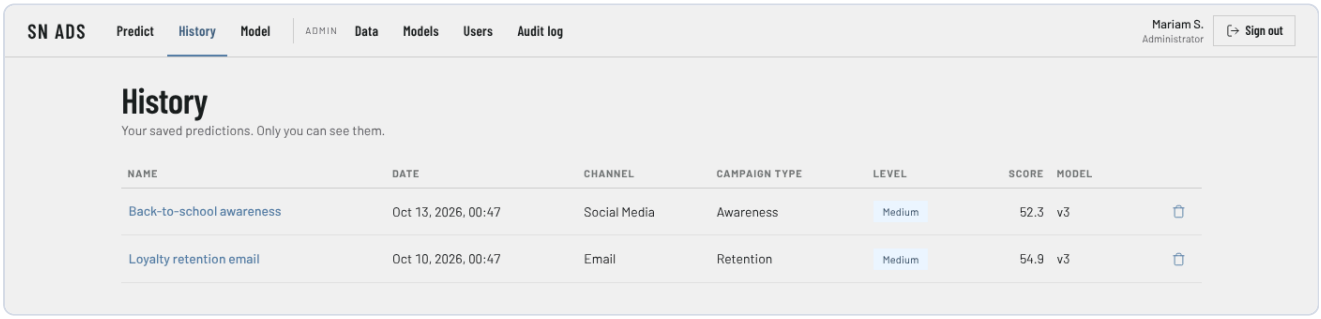
The Predict page: form on the left, results on the right (prototype, illustrative values).

Step 3 — Read the results

What you see	What it means
Click-through rate	The share of people who saw the ad and clicked on it, shown next to the dataset average for comparison.
Conversion rate	The share of visitors who do what the campaign wants (buy, register…).
Website visits	The number of visits the campaign is expected to bring.
Performance level	**Low, Medium or High**, with a score out of 100 and three sub-notes (click rate, estimated leads, estimated cost per result). Each note says how the campaign ranks against *similar* campaigns.
What moved this prediction	For each of the three figures, the five inputs that pushed it up or down the most, with an arrow showing the direction.
Suggested changes	Up to three changes (e.g. “use another channel”, “lower the budget by 20 %”) with the before/after effect. An **Apply** button applies the change and recalculates. If nothing helps: “No tested change improves the result significantly.”
Banners	Warnings: the model has low accuracy; no model is active yet; a value you typed is outside what the model has seen.

Step 4 — Save and come back later (“History”)

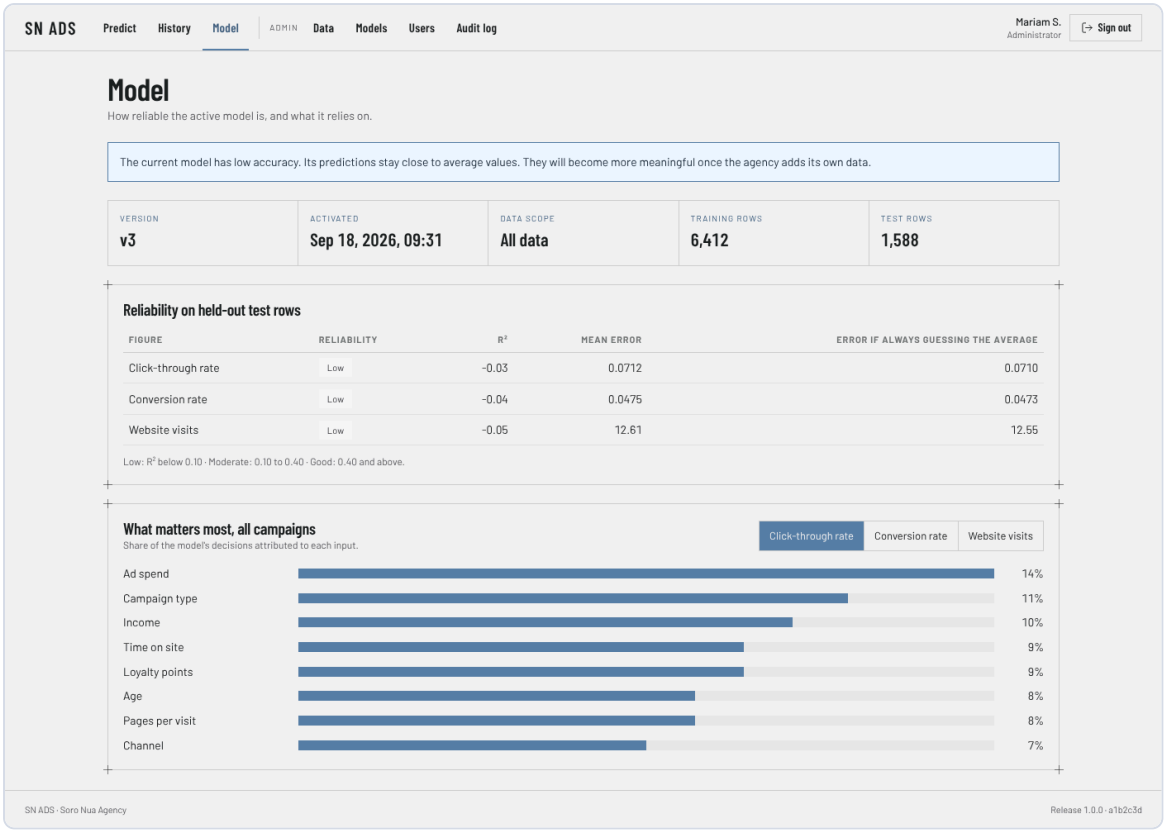
A prediction can be saved with a name. The **History** page lists only **your own** predictions; opening one shows it exactly as it was, even if the model has changed since.



History: every user sees only their own saved predictions.

Step 5 — The “Model” page (for everyone)

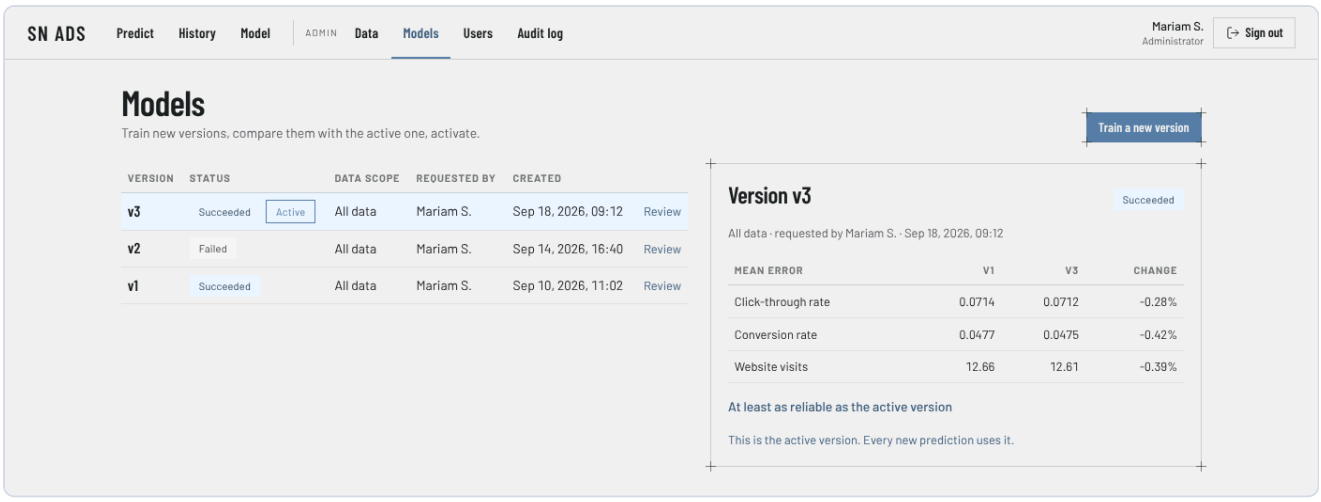
Transparency page: which model version is active, when it was activated, how many rows it learned from and was tested on, how **reliable** it is for each of the three figures, and which inputs matter most across all campaigns.



The Model page: reliability on held-out test rows and importance of each input (prototype).

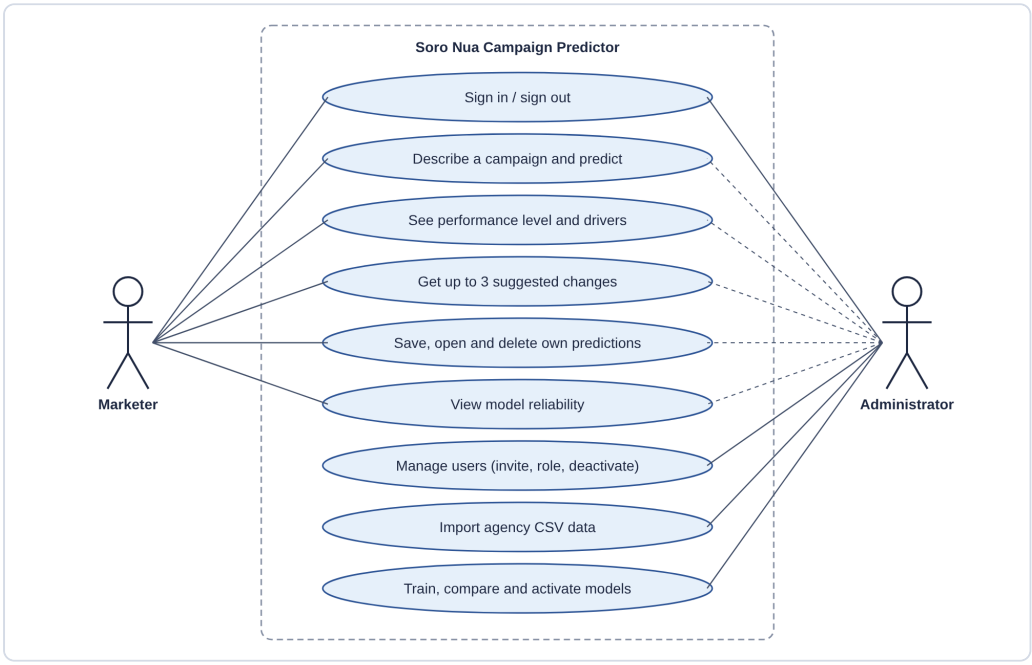
Step 6 — Administrator pages

Page	Purpose
Data	Shows how many starting rows and agency rows exist and how many are kept aside for testing; upload an agency CSV file (the whole file is checked first; any error is listed with its row and column and nothing is imported).
Models	“Train a new version”, follow its status (Queued, Running, Succeeded, Failed), review its measures, compare with the active version, and “Activate this version”.
Users	Invite colleagues, change a role (Marketer / Admin), deactivate or reactivate an account.
Audit log	A read-only, chronological record of sensitive actions (who invited whom, who trained or activated a model, who imported data…).



The Models page (administrator): versions, status and comparison with the active version.

Who can do what



Use-case diagram: the administrator can do everything the marketer can, plus the administration tasks.

4 The key ideas, explained simply

Nothing in the project requires advanced mathematics to understand. Here are the core ideas, each with an everyday comparison.

4.1 Learning from examples (machine learning)

Think of an experienced real-estate agent who can estimate a flat's price after seeing thousands of flats. Nobody wrote down rules; the agent learned patterns from examples. A **machine-learning model** does the same with a table of past cases: for each past campaign we know the inputs (budget, channel, audience…) and the results (clicks, conversions, visits). The model finds the patterns that connect them. Predicting a *number* from inputs is called **regression**.

4.2 XGBoost: a team of small decision trees

A **decision tree** is a chain of yes/no questions ("is the budget above 5,000? is the channel email?") ending in a number. One tree is crude. **XGBoost** builds hundreds of small trees in sequence, each one correcting the mistakes of the previous ones, then adds their answers. It works very well on tables like ours, trains in seconds, and can tell us how much each input contributed to an answer — which is exactly what we need to explain predictions.

We train **three separate models**, one for each figure to predict (click-through rate, conversion rate, website visits), all with the same fixed settings so results are reproducible.

4.3 Never grade a student on the questions they studied

If a student memorises the answers to last year's exam and we test them on the same exam, a perfect mark proves nothing. Likewise we **keep about 20 % of the rows aside** ("test rows"); the model never sees them while learning. We then ask it to predict those rows and compare with the truth. That is the honest measure of reliability.

The choice of test rows is **automatic and permanent**: a row's "fingerprint" (a hash of its origin and identifier) decides whether it is a test row, so a given row is always a test row whatever model version is trained.

4.4 R^2: is the model better than guessing the average?

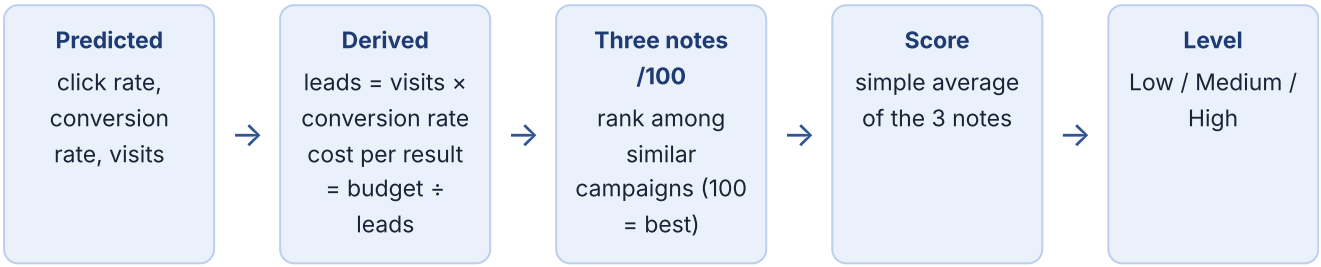
To measure reliability we use a number called **R^2** ("R-squared"):

- **1.0** — perfect predictions;
- **0** — no better than always answering with the average;
- **negative** — worse than the average.

The tool labels reliability **Low** (R^2 below 0.10), **Moderate** (0.10 to 0.40) or **Good** (0.40 and above), and shows the model's average error next to the error of "always guess the average" so anyone can see whether the model adds value.

4.5 Turning three numbers into one rating

A campaign with a high click rate but terrible cost per result is not a good campaign. So the tool computes **three notes out of 100**, then averages them:



- A note is a **ranking**: "this campaign's click rate beats 42 % of the recorded campaigns of the same type". For cost per result, lower is better.
- The comparison is made only against campaigns **of the same type**, so an awareness campaign is not judged by the standards of a conversion campaign. (If a type has fewer than 30 recorded examples, the tool falls back on all types and warns you.)
- The two boundaries between Low, Medium and High are placed so that, among the recorded examples, **one third falls in each level**. They are recomputed at every training.

> **Worked example**
>
> Budget 5,000 USD; predicted 23.5 visits and 11.45 % conversion → **2.69 leads** → cost per result ≈ **1,860 USD**. Notes: click rate 42, leads 59, cost 52 → score (42+59+52)/3 = **51.0**. With boundaries at 41.2 and 58.9 the level is **Medium**. (Illustrative values from the prototype.)

4.6 Explaining a prediction

For every prediction the model can say how much each input pushed the figure above or below its baseline. The tool shows the **five strongest**, with an arrow. In the Model page the same idea is averaged over all campaigns to show which inputs matter globally. This is the principle of *Shapley values* (a fair way of splitting credit between inputs).

4.7 Suggesting improvements: "what if?"

The tool does not guess advice; it **tests alternatives**. For one decision at a time (the "levers") it tries other values, re-predicts, re-scores, and keeps the best:

Lever	Values tried
Budget	from −50 % to +50 % in steps of 10 %

Channel	each of the 4 other channels
Campaign type	each of the 3 other types
Age, income	other bands (e.g. 21, 30, 40, 50, 62 years)
Gender, past purchases, loyalty points	the other gender; other levels

About 40 variants are evaluated in one batch. For each lever only the best variant is kept; it is proposed **only if it raises the score by at least 5 points out of 100**; the three best are shown. “Expected engagement” is never a lever, because it is a *consequence* of the campaign, not a decision.

4.8 Row-level security (the database as a bodyguard)

Instead of trusting every line of application code to check “is this user allowed to see this row?”, the **database itself** refuses to return rows the user may not see. The rule is written once, in the database, and tested. Even if the application code had a bug, a marketer still could not read a colleague's predictions.

5 The data

5.1 The starting dataset

The agency has no campaign history of its own yet, so the model was first taught from a public dataset, *"Predict Conversion in Digital Marketing Dataset"* (Kaggle): **8,000 rows, 20 columns**. It is stored in the repository (`data/seed/`) with a note on its origin and a checksum, and is loaded into the database once.

Group	Columns	Used as
Campaign plan (known before launch)	CampaignChannel (Email, PPC, SEO, Social Media, Referral); CampaignType (Awareness, Consideration, Conversion, Retention); AdSpend (≈ 100 to 10,000)	Input and lever
Target profile	Age (18–69), Gender, Income (≈ 20k–150k), PreviousPurchases, LoyaltyPoints	Input and lever
Expected engagement (known after launch)	PagesPerVisit, TimeOnSite, SocialShares, EmailOpens, EmailClicks	Input (pre-filled), never a lever
To predict	ClickThroughRate (0.01–0.30), ConversionRate (0.01–0.20), WebsiteVisits (0–49)	Three targets
Ignored	CustomerID (just a row number); Conversion (an outcome, not a description); AdvertisingPlatform and AdvertisingTool (same value on every row)	—

5.2 The uncomfortable truth about this dataset

> **Synthetic data with no signal**
>
> An analysis done early in the project (three notebooks, see section 11) showed that this dataset is **fabricated**: values are almost uniformly random, every row describes a person rather than a campaign, and — crucially — the three figures to predict have **no exploitable relationship** with the other columns (absolute correlation below 0.025).

Consequences, all verified by measurement and all displayed honestly in the tool:

- On the 1,602 test rows, R^2 is about **−0.03** for each figure: no better than the average. Reliability is labelled **Low** and a banner says so.
- Most campaigns land in the **Medium** level, and advice tends to concentrate on the budget lever.
- To make sure the software itself is not at fault, the same training chain was tried on a synthetic dataset **with a deliberately planted relationship**: it reached R^2 above 0.9. The code learns real patterns when they exist; the limit comes from the data.

5.3 When the agency's own data arrives

An administrator uploads a CSV file with **exactly the same 20 columns** (one row per campaign, described by the typical profile of its target and by its real results). Before anything is stored, the **whole** file is checked:

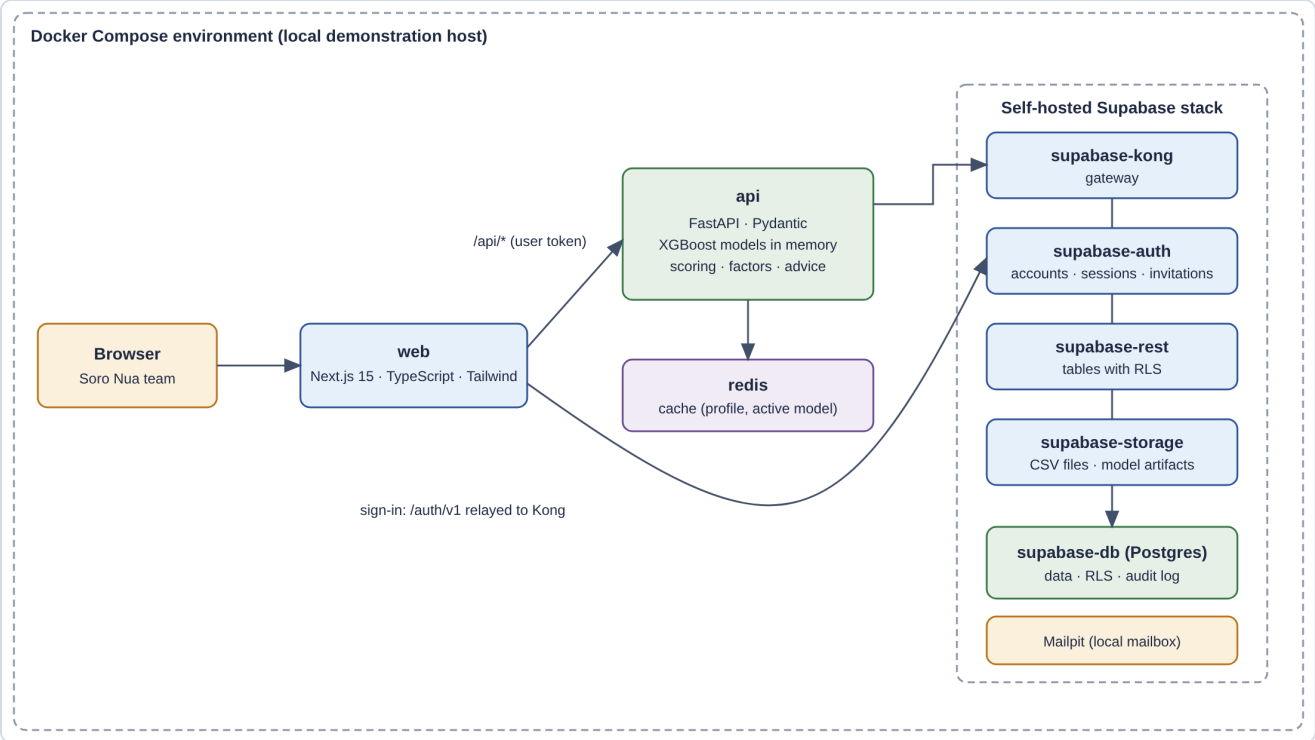
- exact column names, correct types, allowed values and numeric bounds (e.g. rates between 0 and 1);
- no duplicate identifiers; the same file cannot be imported twice;
- cells that could run formulas when opened in a spreadsheet (starting with `=`, `+`, `-`, `@`) are refused;
- the administrator must tick “I confirm that client names are anonymized in this file.”

If anything is wrong, nothing is imported and the first 100 problems are listed with row, column and reason. Each stored row remembers its **origin** (starting set or agency). A training can then use *all data*, or *agency data only* once there are at least 250 agency rows.

6 How the system is built (the big picture)

6.1 The principle: everything in containers

A **Docker container** is a sealed box holding one program and everything it needs. Every part of the project runs in such a box, so it behaves the same on any machine, and nothing has to be installed by hand. One command (`./start.sh`) starts the whole system.



Architecture of the local demonstration environment.

6.2 The parts, in plain words

Box	Technology	What it does
web	Next.js, TypeScript, Tailwind	The website the user sees: pages, forms, charts. It never touches the database directly.
api	Python, FastAPI, XGBoost	The “brain”: checks who you are, validates inputs, runs the prediction, scoring, explanations and advice. It keeps the active model in memory so answers are fast.
supabase-db	PostgreSQL	The database: users' profiles, the dataset rows, model versions, saved predictions, audit log.
supabase-auth	Supabase Auth	Accounts, passwords, invitations, sign-in sessions.
supabase-rest	PostgREST	Gives controlled access to tables — always applying the security rules for the signed-in user.

supabase-storage	Supabase Storage	Keeps files: uploaded CSVs and the saved model files.
supabase-kong	Kong gateway	The single entrance to the Supabase group.
redis	Redis	A fast memory: caches user profiles (60 s) and remembers which model version is active.
mailpit	Mailpit	A local mailbox that catches invitation and password-reset e-mails during the demonstration.

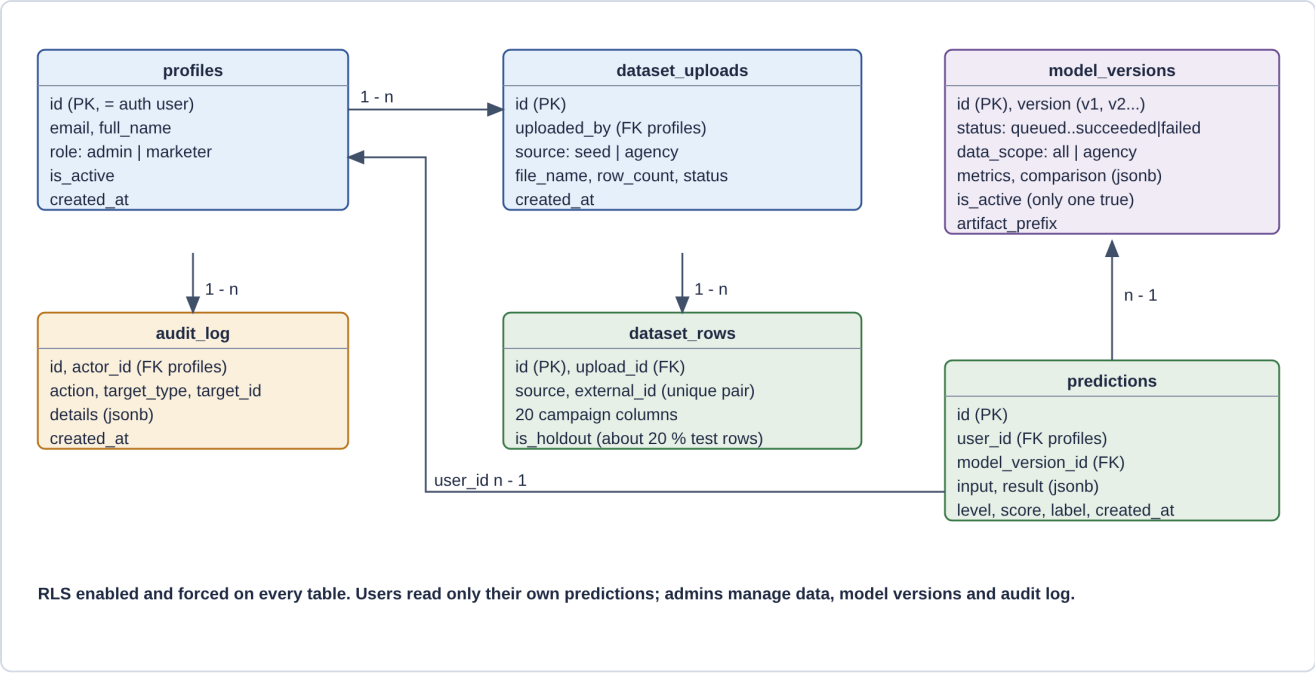
Supabase is an open-source toolbox (database + sign-in + file storage). We run our own copy inside the containers instead of using the paid online service, which keeps the agency's data under its own control.

6.3 The golden rules of the architecture

- **The browser only talks to the website.** The website forwards `/api/*` calls to the api and sign-in calls to the Supabase gateway. The browser never queries tables.
- **The api acts as the user.** It queries the database with the user's own token, so the database security rules apply to every request.
- **The "master key" is rarely used.** The powerful Supabase service key exists only in a short, closed list of modules (user administration, model registry, command-line commands) and never in the website or the browser.
- **One code path for learning and serving.** The same code that trains the model is the code that serves it, so nothing can drift apart.

6.4 The data model

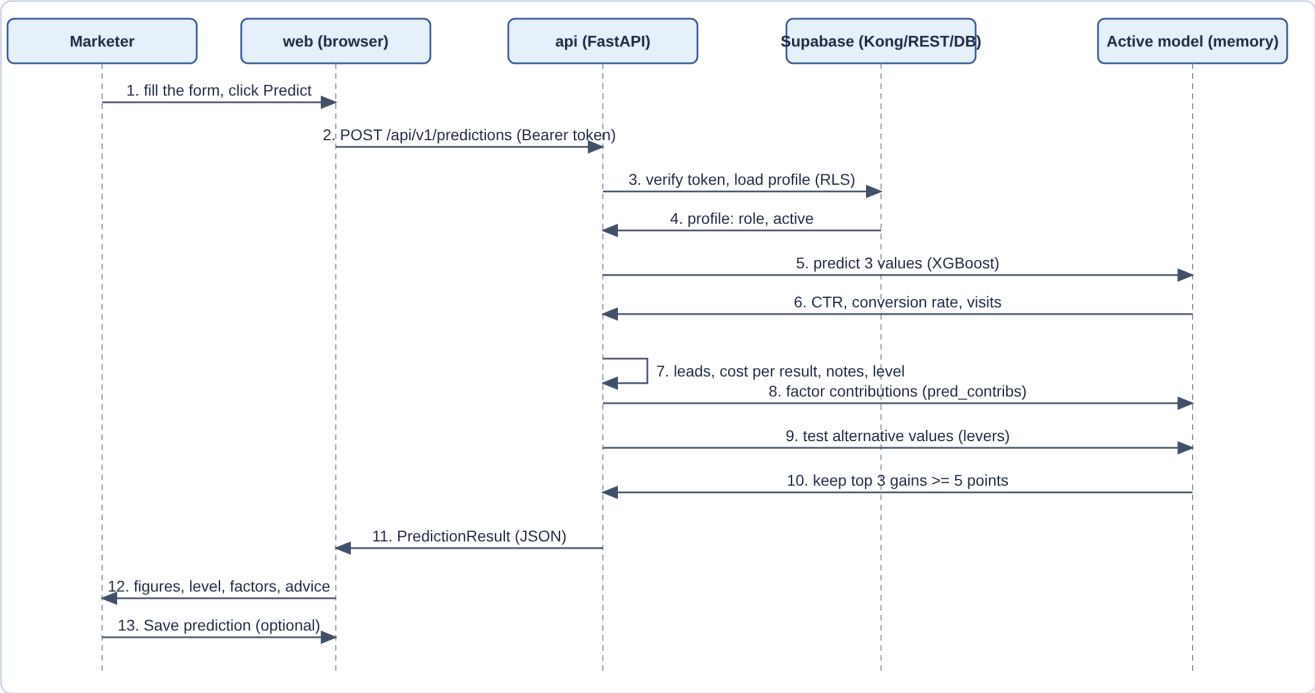
The database has six tables. Row-level security is **enabled and forced** on every one of them.



Data model: who owns what.

Table	Contents
`profiles`	One row per user: e-mail, name, role (admin or marketer), active or not.
`audit_log`	Who did what, when, to what (sensitive actions only).
`dataset_uploads`	One row per import (starting set or agency file): who, when, how many rows, status.
`dataset_rows`	The campaign rows themselves, with their origin and whether each is a test row.
`model_versions`	Each trained version: status, data scope, measured reliability, importance of inputs, where its files are, and whether it is the active one (only one can be).
`predictions`	Saved predictions: the input, the full result, level, score and the model version used — frozen once saved.

6.5 The life of one prediction



What happens between clicking “Predict” and seeing the results.

7 How every feature works, step by step

7.1 Signing in and roles

1. An administrator invites a colleague (e-mail, full name, role). The system sends an invitation e-mail (caught by the local mailbox in the demonstration).
2. The colleague opens the link and sets a password. Supabase Auth then issues a **token**: a signed digital pass that proves who they are for a limited time.
3. Every request to the api carries this token. The api checks the signature, expiry, audience and issuer, then loads the user's **profile** (role, active or not) from the database — cached for 60 seconds.
4. If the profile says “deactivated”, access is refused within a minute, even for someone already signed in.

Safeguards: an administrator cannot change or deactivate their *own* account, and any action that would leave **zero active administrators** is refused.

7.2 Loading the starting data

A command-line command (not a web page) reads the CSV, validates it, and inserts the 8,000 rows once. For each row it computes the permanent “test row” flag (about one row in five). Running the command twice changes nothing: **operations are idempotent**.

7.3 Training the three models

1. The administrator clicks **Train a new version** and chooses the scope (*All data* or *Agency data only*). A new version row is created with status **Queued**. Only one training can run at a time.
2. The system reads the rows of that scope and separates **training rows** (about 80 %) from **test rows** (about 20 %).
3. Text values (channel, type, gender) are converted to numbers by standard encoding; numeric columns are kept as they are.
4. Three **XGBoost** models are trained independently (300 trees, depth 4, learning rate 0.05, fixed random seed 42).
5. Each model is **evaluated on the test rows**: R^2, average error, and the error of the “always average” baseline. This gives the Low / Moderate / Good label.
6. The system prepares what the tool needs at prediction time: the **reference tables** for the notes (sorted real values per campaign type), the **level boundaries** (one third / one third / one third), the **average engagement** per channel and type for the form's pre-filled fields, the **input ranges** seen in training, and the **global importance** of each input.

7. Models and references are saved as files in storage; the version becomes **Succeeded** (or **Failed** with a readable reason, e.g. “Not enough data: at least 200 training rows and 50 test rows are required.”).
8. The administrator reviews the measures, compares with the active version, and clicks **Activate this version**. Only one version is active at a time; the api reloads it within seconds.

> **Demonstration vs. full design**
>
> The full design runs training in a separate *worker* container fed by a queue, and blocks activation of a version less reliable than the current one. In the demonstration, training runs inside the api container and comparison/activation is simplified (see section 13).

7.4 Predicting

1. The browser validates the form (the same bounds as the server) and sends it to the api.
2. The api validates it again, strictly: any unknown or out-of-range field is rejected with a clear message. **The result computed by the browser is never trusted**; everything is recalculated on the server.
3. The active model predicts the three figures; outputs are bounded to valid ranges (rates between 0 and 1, visits not negative).
4. The api derives estimated leads and cost per result, computes the three notes, the score and the level.
5. It computes the top-five contributions for each figure, and builds the list of warnings.
6. It tests the ~40 variants for the advice and keeps up to three gains of 5+ points.
7. One JSON result goes back to the page, which draws the figures, the level, the factor bars and the advice cards.

7.5 Saving and history

“Save prediction” asks for a name and sends the *input* to the server, which **recomputes** the prediction and stores input + result + model version. A saved prediction is never edited. The History page lists only the signed-in user's rows, newest first. Asking for another user's prediction returns *“Prediction not found.”* — never “forbidden”, so its existence is not even revealed.

7.6 Importing agency data and retraining

See section 5.3 for validation. After a successful import the Data page shows the new counts (“Agency data: 400 — enough for ‘Agency data only’”). The administrator then trains a new version, reviews it, and decides whether to activate it. Two small sample files are included to try this: one valid (400 campaigns) and one with deliberate errors, to see the error list.

7.7 Error handling

Every error in the whole project uses **one standard format**: a stable *code* (e.g. `NOT_FOUND`, `VALIDATION_ERROR`, `NO_ACTIVE_MODEL`), a human-readable English *message*, optional *details* (for a CSV: row, column, reason) and a *request_id* shown as “Reference: …” so the problem can be traced. Users never see a technical trace.

8 Security and privacy

Security was written down **before** building (a dedicated document), and applied as a rule on every route and table.

Risk	How it is handled
Seeing another user's data	Row-level security enabled *and forced* on every table, with a positive and a negative automated test per policy. Others' resources always answer 404.
Becoming administrator illegitimately	The role is read from the database, never from the browser. Users cannot edit their own profile row.
Misuse of the master key	Service key usable only by a closed list of modules; never in the website image or the browser.
Direct attacks on the database	Only the website, `/api/*` and `/auth/v1/*` are meant to be reachable; tables and storage are not exposed.
Malicious or broken input	Strict validation (Pydantic, unknown fields forbidden; Zod in the browser); size limits (10 MB files, 64 KB JSON).
Spreadsheet formula injection	CSV cells starting with `=` `+` `-` `@` are refused.
Trusting the interface	Predictions, scores and roles from the browser are never trusted; all recomputed server-side.
Leaking secrets	No secret in code, tests, logs or Git. A secret scanner runs before each commit. Configuration is read in one place only.
Abuse	Per-user rate limits (e.g. 120 predictions/minute, 5 trainings/hour); generic sign-in failure message; security headers.
Accountability	An audit log of sensitive actions, readable by administrators only.

> **Privacy by design**
>
> The starting dataset contains no real personal data. For agency files, the administrator must confirm that client names are anonymized before importing. No external AI service receives any data: nothing leaves the agency's own containers.

9 Our approach: how the work was organised

The user of this project is a marketing team, not developers. The way of working was designed so that the result is **predictable, verifiable and explainable**.

1. Documents first

Thirteen reference documents were written before the code: brief, product requirements (PRD), architecture, data model, API specification, low-level design, conventions, environment setup, testing strategy, security notes, CI/CD, roadmap and a decisions log. If two documents disagree, a priority order says which wins (brief > security > data model > API > design).

2. Tests before code

For each behaviour: write a test, watch it **fail**, write the code until it passes, then tidy up. This proves each test can actually detect a problem, and every feature is protected against future breakage.

3. Small phases, visible results

The roadmap splits the project into phases (0 to 20). Each phase ends with something you can **see on screen** and a checklist of "done" criteria.

4. A decisions log

Whenever a choice was not covered by the documents (a library version, a naming rule, a trade-off), it was decided using a fixed order — security by default, simplicity, consistency, most standard solution — and recorded with date, reason and documents updated. There are 100+ entries.

5. Pinned versions

Every dependency is fixed to an exact version, and security audits run without ignoring warnings, so the build is reproducible and known-vulnerable versions are replaced.

6. Everything in containers, via the Makefile

No tool is installed on the host machine. All checks, tests, migrations and servers run in the project's images through `make` commands.

The three-branch rule

Code flows through three protected branches: **dev** (construction) → **staging** (validation) → **main** (production). Work for a phase happens on its own branch, reviewed and merged into dev; promotion upwards is only by pull request. Direct pushes to the protected branches, forced pushes and rewriting shared history are forbidden.

Demonstration mode

A scoping decision

The project is presented as a **local demonstration**. To focus on what matters for the demonstration, some phases are out of scope (a separate training worker and queue, live recalculation while typing, cloud deployment on AWS, automated delivery pipelines, error-monitoring service, real e-mail delivery, production release). In this mode some process rules are lighter (no per-phase pull-request ceremony, no coverage threshold), but the essential safeguards remain **mandatory**: row-level security with tests on every table, database access with the user's token, 404 for other people's resources, the standard error format, no secret in Git, exact versions, everything in Docker, all labels in one file.

10 The steps taken, phase by phase

The internship ran for ten weeks. The work followed the roadmap, which itself follows a deliberate order: **foundations → authentication → data → features → (later) external integrations**.

Phase	Title	What it delivered / what you can see	Status
0	Repository foundation	A tidy repository that starts with one command: API with a health page, website showing "API: healthy" (green) or "API: unreachable" (red), Docker setup, Makefile, pre-commit checks, test tools configured, starting dataset stored with its source note.	done
1	Local database and profiles	The self-hosted Supabase stack running in containers; `profiles` and `audit_log` tables with row-level security and database tests; the website shows "Database: connected (schema number)".	done
2	Authentication	Sign in, sign out, set password from an invitation, forgot password; protected pages; header with the user's name and role.	done
3	User management and audit	Administrator pages to invite users, change roles, deactivate/reactivate, and read the audit log; "last administrator" and "own account" safeguards.	done
4	Data storage and starting dataset	Tables for dataset rows; command that loads the 8,000 rows once; Data page with counts.	done (demo)
5	Agency data import	Upload and full validation of agency CSV files; clear error lists. (Initially out of the demo scope; added afterwards, including agency-only training.)	added later
6	Queue and worker	Separate training worker fed by a queue.	not in demo
7	Training the three models	Train, evaluate on test rows, store versions; Models page; reliability labels.	done (demo)
8	Comparison, activation, Model page	Before/after comparison and rule-based activation. In the demo: review and an "Activate" button.	simplified
9	Form and prediction	The three-block form; the three predicted figures; validation front and back.	done (demo)
10	Performance level	Estimated leads, cost per result, notes, score and Low/Medium/High.	done (demo)
11	Campaign factors	Top-five factors per figure with direction; global importance on the Model page.	done (demo)

12	Recommendations	Up to three changes with before/after and an Apply button; clear message when none helps.	done (demo)
13	Live simulation and draft	Recalculation while typing; form restored after session expiry.	not in demo
14	Save, history, isolation	Save with a name, History page, open and delete; proof that users cannot see each other's predictions.	done (demo)
15–20	Robustness, monitoring, AWS, CI/CD, real e-mails, production	Full end-to-end suite, Sentry error monitoring, AWS infrastructure (Fargate, load balancer, secrets), automated pipelines, Amazon SES, production release with manual approval.	not in demo

Extra tracks done around the phases

- **Analysis notebooks** (preparation of phase 7): three Jupyter notebooks, run in Docker, that profile the data, train the three models and run **honesty checks** (shuffled target, cross-validation, a dataset with planted signal). They produced the measured facts recorded in the decisions log (1,602 test rows, $R^2 \approx -0.03$, level boundaries around 40 and 61).
- **Interface conformity**: the web application was aligned with the interface prototype supplied for the project, and served in production mode so pages open quickly.
- **One-command launcher**: `start.sh` / `make demo` to run the whole demonstration, plus a six-step demonstration route in the README.

The ten internship weeks at a glance

Week	Main focus
1 (22–26 Jun)	Discovery of the agency, understanding the problem and the users
2 (29 Jun – 3 Jul)	Requirements, brief, product requirements, choice of stack, architecture documents
3 (6–10 Jul)	Study of the dataset and first models in notebooks; honesty checks
4 (13–17 Jul)	Repository foundation, Docker, API and website skeletons, Supabase stack
5 (20–24 Jul)	Authentication, user management, audit log, row-level security
6 (27–31 Jul)	Dataset loading, training of the three models, evaluation
7 (3–7 Aug)	Form, predictions, derived figures and performance level
8 (10–14 Aug)	Factors and recommendations
9 (17–21 Aug)	History, isolation, CSV import, interface alignment, demo launcher
10 (24–27 Aug)	Final tests, documentation, demonstration, report

The weekly split is a reconstruction following the order of the project phases.

11 Quality: how we know it works

11.1 Layers of tests

Each kind of risk has its own kind of test, run in containers through the Makefile:

Level	Tool	What it protects
API unit tests	pytest	The business rules: CSV validation, test-row rule, derived figures, notes and score, advice engine, configuration, error format, and "architecture guards" (e.g. the service key is only obtained where allowed). Around 600 test functions in total for the API.
API integration tests	pytest + FastAPI test client	Every route: authentication, roles, validation, status codes, isolation between users, upload and model routes.
Database tests	pgTAP	15 test files: every security policy (allowed and refused cases), triggers (last administrator), privileges.
Website unit tests	Vitest, Testing Library, MSW	Form validation, formatting, API client, display components, role guard.
End-to-end tests	Playwright	Real browser journeys: sign-in, sessions, administrator users page, prediction journey, isolation between users, the demonstration route.

11.2 Honesty checks on the model

Beyond "does the code work?", we asked "is the model *honestly* good?". In the notebooks we trained on a **shuffled** target (should give nothing), ran **cross-validation** (stability), and trained on a dataset with a **planted signal** (should give R^2 above 0.9). All behaved as expected, which proves the pipeline is sound and that the low reliability comes from the data.

11.3 Guard rails against silent mistakes

- The measured reference values (1,602 test rows, level boundaries around 40 and 61) were recorded early, so that a suspiciously *good* model later (a sign of data leakage) would be noticed.
- A deliberately failing placeholder (`make test-db` refusing to "pass" when migrations exist but no database test does) forced real tests to replace a fake green.
- Security audits (`pip-audit`, `npm audit`) run without ignoring findings, which led to several dependency upgrades, each logged.

12 Running and demonstrating the project

12.1 What you need

A computer with **Docker Desktop** (at least 6 GB of memory allocated to Docker), **git** and **make** (on Windows: WSL or Git Bash). Nothing else.

12.2 Start it

Command	What it does
`./start.sh`	The simplest way: does everything below for you.
`make env-local`	Creates the local configuration files and generates local secrets (never overwrites a real value).
`make build`	Builds the images and pulls the pinned Supabase images (several minutes the first time).
`make demo`	Starts the application, loads the 8,000 starting rows, creates two demonstration accounts, trains and activates a first model. Safe to run again.
`make test-db` / `make test-e2e`	Runs the database tests / the browser end-to-end scenarios.
`make notebooks-run`	Runs the three analysis notebooks.
`make db-reset`, `make clean`	Rebuild the database / wipe everything and start again.

The application opens at **http://localhost:3000**. The e-mail/password of the two demonstration accounts (an administrator and a marketer) are written in the file `.demo-accounts.txt`, created by `make demo` and never published.

12.3 The six-step demonstration route

1. **Sign in as administrator.** The menu shows Predict, History, Model, Data, Models, Users and Audit log.
2. **Data:** “Seed rows: 8,000” and the number of test rows kept aside.
3. **Models:** “Train a new version”; watch the status go Queued → Running → Succeeded; open it to see the measures; “Activate this version”.
4. **Model:** active version, its reliability (Low, with a warning) and the global importance chart.
5. **Predict:** describe a campaign, click “Predict”; read the figures, level, factors (tabs) and suggestions; “Apply” a suggestion to see the new result.

6 **Save, then History:** save with a name; list, open (“exactly as it was”), delete.

Then sign out and sign in as the **marketer**: no administration menu, and opening another user's prediction answers “Prediction not found.”

12.4 Trying the data import (optional)

As administrator, Data → “Upload CSV” with `data/sample/agency_sample_400.csv`: tick the anonymization confirmation, upload → “400 rows imported.” Upload it again → “This file has already been imported.” Then Models → “Train a new version” now offers **Agency data only**. The file `agency_sample_invalid.csv` shows the error list (a click rate of 5, a cell starting with “=”).

12.5 If something goes wrong

- **A port is already used** (3000, 8000, 54321, 54323, 54324): stop the other program or change the port in `.env`.
- **Docker is slow or containers keep restarting:** give Docker more memory, close other projects, then `make down` and `make demo`.
- **“No active model yet”:** run `make demo` again; it trains and activates a version if needed.
- **Everything from scratch:** `make clean`, then `make demo`.

13 Honest limits and what comes next

Limit	Explanation	Next step
Synthetic starting data	No relationship between inputs and results; reliability Low; advice dominated by budget.	Import the agency's real campaign data (≥ 250 campaigns) and retrain.
One row per person	The starting file describes persons, not campaigns; advertising platform masked.	Agency data: one row per campaign, platform filled in.
Training inside the api	No queue or separate worker in the demonstration.	Dedicated worker with a queue.
Simplified model comparison	No automatic rule blocking a less reliable version.	Automatic comparison on the same test rows, activation only if at least as reliable.
No live recalculation	Results refresh on the Predict button, not while typing.	Live simulation with a short typing pause; draft restoration.
Local only	No cloud deployment, real e-mail delivery or monitoring service.	AWS deployment, CI/CD pipelines, Amazon SES e-mails, Sentry.
Narrow advice	Only decisions the agency really takes: budget, channel, type, target profile.	Content or platform advice if such data exists. One lever at a time today; combinations later.

Why the project is still valuable

The hard, reusable part is done: a secure, tested, reproducible application with a validated learning pipeline, transparent reliability, explanations and advice. The day the agency's real data is imported, the same system starts learning real relationships — with no change to the code.

14 Questions people ask

“If the predictions are no better than the average, why build this?”

Because the project proves the whole chain works and is safe, and because the weakness is in the *data supplied at the start*, not in the method. A planted-signal test shows the pipeline reaches R^2 above 0.9 when real relationships exist. The tool also says clearly when it cannot be trusted, which is better than showing impressive but meaningless numbers.

“Does it need the internet?”

No. In the demonstration everything runs on the local machine. Internet is only needed once, to download the images when building.

“Who can see my saved predictions?”

Only you. This is enforced by the database itself, not only by the application. Administrators manage users, data and models but have no menu to browse other people's predictions.

“Can a bad model replace a good one by mistake?”

A new version never becomes active on its own: an administrator must explicitly activate it after reviewing its measures. The full design adds an automatic block on versions that are less reliable than the current one.

“Why XGBoost rather than deep learning or a language model?”

For a table with a few thousand rows, boosted trees are accurate, fast, stable, and can explain their answers. Generative AI was deliberately kept out of the first version: it would add cost and risk without value while the data is synthetic.

“What does a 'Medium' really mean?”

It is relative: the campaign ranks in the middle third of the recorded campaigns *of the same type* on a mix of click rate, leads and cost per result.

“Why so many documents and tests for a small tool?”

Because the person who will use it is not a developer and cannot check the code. The documents, tests and the decisions log are what make the result checkable by someone else.

“How do I add a new campaign type or channel?”

Those values are part of the data format (the same in the starting dataset and in agency files) and of the form options. Adding one is a change of the data contract and of the training: it is a deliberate project change, planned first, then done through the documents and tests, not a quick edit.

15 Glossary and map of the repository

Glossary

Term	Meaning
API	The door through which programs (here, the website) ask the “brain” to do things.
Audit log	A record of who did which sensitive action and when.
Click-through rate (CTR)	Share of people who saw an ad and clicked.
Conversion rate	Share of visitors who complete the wanted action.
Container / Docker	A sealed box running one program with all it needs; Docker runs these boxes.
CSV	A simple table saved as text, one row per line, columns separated by commas.
Endpoint / route	One address of the API doing one job (e.g. saving a prediction).
Estimated leads	Visits × conversion rate: the expected number of interested prospects.
Idempotent	Safe to run twice: the second run changes nothing.
JWT / token	A signed digital pass proving who you are for a limited time.
Lever	A decision the agency can really change (budget, channel, type, target profile) and that advice may modify.
Migration	A versioned, ordered change of the database structure; once applied it is never edited.
Model version	One trained set of three models, with its measures and files; only one is active.
pgTAP	A tool to write automated tests inside PostgreSQL.
Playwright	A tool that drives a real browser to test the website like a user.
R^2	Reliability measure: 1 perfect, 0 as good as the average, negative worse.
RLS	Row-level security: the database decides which rows each user may see.
Test rows (holdout)	About 20 % of the rows never used for learning, kept to measure reliability.
TDD	Test-driven development: write the test first, then the code.
XGBoost	A learning method that builds many small decision trees, each correcting the previous ones.

Map of the repository

Folder / file	What you find there
`CLAUDE.md`	Permanent working rules for the development assistant (rules, stack, definition of done).
`docs/`	The thirteen reference documents and the decisions log. Start with `PROJECT_BRIEF.md`, then `ROADMAP.md`.
`apps/api/`	The Python service: routes, business rules (`domain/`), learning code (`ml/`), database access (`repositories/`), command-line commands (`cli/`), tests.
`apps/web/`	The Next.js website: pages, components, the labels file (all on-screen text), unit and end-to-end tests.
`supabase/`	The database migrations (SQL) and the pgTAP tests.
`deploy/supabase/`	The self-hosted Supabase stack (Docker Compose, gateway config, key generation).
`data/seed/` , `data/sample/`	The starting dataset with its source note; two small sample agency files.
`notebooks/`	The three analysis and training notebooks.
`ui_prototype/`	The interface prototype used as visual reference.
`Makefile` , `start.sh` , `docker-compose*.yml`	The commands and container definitions that run everything.
`README.md`	Quick start and the demonstration route.
`report/`	The academic internship report and this guide.

> **In one sentence**
>
> Soro Nua Campaign Predictor learns from past campaigns to estimate the results of a new one, rates it, explains why and suggests improvements — inside a secure, tested, container-based application — and is honest about the one thing it still needs: the agency's own real data.